

LTE Standard(U)系列

QuecOpen RTC API 参考手册

LTE Standard 模块系列

版本：1.1

日期：2023-08-23

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不

侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更表述
-	2021-07-19	Neo KONG	文档创建
1.0	2021-08-03	Neo KONG	受控版本
1.1	2023-08-23	Neo KONG/ Zack TAN	1. 新增适用模块 EC200D 系列、EC200G-CN、EC600G 系列、EC800G-CN、EG700G-CN、EG800G 系列、EG912U-GL 和 EG915U 系列。 2. 新增 EC200D 系列模块文件路径的备注(第 1.1 章)。 3. 新增枚举 <code>ql_errcode_rtc_e</code> 的成员 <code>QL_RTC_SET_CFG_ERR</code> (第 2.3.1.1 章)。 4. 新增函数 <code>ql_rtc_get_cfg()</code>和 <code>ql_rtc_set_cfg()</code>，以及结构体 <code>ql_rtc_cfg_t</code> (第 2.3.9 章和第 2.3.10 章)。 5. 新增不同模块支持的 log 工具相关备注(第 3.2 章)。

目录

文档历史	3
目录	4
表格索引	5
图片索引	6
1 引言	7
1.1. 适用模块	7
2 RTC API	9
2.1. 头文件	9
2.2. 函数概览	9
2.3. 函数详解	10
2.3.1. ql_rtc_set_time	10
2.3.1.1. ql_errcode_rtc_e	10
2.3.1.2. ql_rtc_time_t	11
2.3.2. ql_rtc_get_time	12
2.3.3. ql_rtc_print_time	12
2.3.4. ql_rtc_set_alarm	12
2.3.5. ql_rtc_get_alarm	13
2.3.6. ql_rtc_enable_alarm	13
2.3.7. ql_rtc_register_cb	14
2.3.7.1. ql_rtc_cb	14
2.3.8. ql_rtc_get_localtime	14
2.3.9. ql_rtc_get_cfg	15
2.3.9.1. ql_rtc_cfg_t	15
2.3.10. ql_rtc_set_cfg	16
3 示例	17
3.1. 开发示例	17
3.2. 功能调试	19
4 附录 参考文档及术语缩写	21

表格索引

表 1: 适用模块	7
表 2: 函数概览	9
表 3: 参考文档	21
表 4: 术语缩写	21

图片索引

图 1: ql_rtc_app_init()	17
图 2: ql_rtc_demo_thread().....	18
图 3: ql_init_demo_thread()	19
图 4: COM 设备图 1 (ECx00U 系列/EGx00U 系列/EG91xU 系列模块)	19
图 5: COM 设备图 2 (EC200D 系列/ECx00G 系列/EGx00G 系列模块)	20
图 6: RTC Log 信息.....	20

1 引言

移远通信 LTE Standard(U)系列模块支持 QuecOpen®方案。QuecOpen®是基于 RTOS 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]和[2]。

本文档主要介绍在 QuecOpen®方案下，LTE Standard(U)系列模块的 RTC API、RTC 开发示例及相关调试。

1.1. 适用模块

表 1: 适用模块

模块系列	模块
-	EC200D 系列
ECx00G	EC200G-CN
	EC600G 系列
	EC800G-CN
ECx00U	EC200U 系列
	EC600U 系列
EGx00G	EG700G-CN
	EG800G 系列
EGx00U	EG500U-CN
	EG700U-CN
EG91xU	EG912U-GL
	EG915U 系列

备注

对于 EC200D 系列模块，文档中所涉及文件路径的路径名称中均不包含 *components*。

2 RTC API

2.1. 头文件

RTC API 头文件为 `ql_api_rtc.h`，位于 SDK 包的 `components\ql-kernel\inc` 目录下。若无特别说明，本文档所涉头文件均在该目录下。

2.2. 函数概览

表 2：函数概览

函数	说明
<code>ql_rtc_set_time()</code>	设置 RTC 时间
<code>ql_rtc_get_time()</code>	获取当前 RTC 时间
<code>ql_rtc_print_time()</code>	打印 RTC 时间
<code>ql_rtc_set_alarm()</code>	设置警报时间
<code>ql_rtc_get_alarm()</code>	获取警报时间
<code>ql_rtc_enable_alarm()</code>	启用警报功能
<code>ql_rtc_register_cb()</code>	注册警报回调函数
<code>ql_rtc_get_localtime()</code>	获取本地时间（含时区）
<code>ql_rtc_get_cfg()</code>	获取 RTC 启动配置
<code>ql_rtc_set_cfg()</code>	设置 RTC 启动配置

2.3. 函数详解

2.3.1. ql_rtc_set_time

该函数用于设置 RTC 时间。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_set_time(ql_rtc_time_t *tm)
```

- 参数

tm:

[In] RTC 时间结构体指针；详见第2.3.1.2 章。

- 返回值

详见第2.3.1.1 章。

2.3.1.1. ql_errcode_rtc_e

RTC 功能组件的结果码枚举定义如下：

```
typedef enum
{
    QL_RTC_SUCCESS = QL_SUCCESS,
    QL_RTC_INVALID_PARAM_ERR = 1|QL_RTC_ERRCODE_BASE,
    QL_RTC_SET_TIME_ERROR,
    QL_RTC_SET_CB_ERR,
    QL_RTC_ENABLE_ALARM_ERR,
    QL_RTC_REMOVE_ALARM_ERR,
    QL_RTC_SET_CFG_ERR
}ql_errcode_rtc_e
```

- 成员

成员	描述
<i>QL_RTC_SUCCESS</i>	函数执行成功
<i>QL_RTC_INVALID_PARAM_ERR</i>	输入无效参数
<i>QL_RTC_SET_TIME_ERROR</i>	设置 RTC 时间错误

<i>QL_RTC_SET_CB_ERR</i>	注册回调函数错误
<i>QL_RTC_ENABLE_ALARM_ERR</i>	启用警报错误
<i>QL_RTC_REMOVE_ALARM_ERR</i>	删除警报错误
<i>QL_RTC_SET_CFG_ERR</i>	设置 RTC 启动配置错误

2.3.1.2. ql_rtc_time_t

RTC 时间结构体定义如下：

```
typedef struct ql_rtc_time_struct {
    int tm_sec;
    int tm_min;
    int tm_hour;
    int tm_mday;
    int tm_mon;
    int tm_year;
    int tm_wday;
}ql_rtc_time_t
```

● 参数

类型	参数	描述
int	<i>tm_sec</i>	秒。范围：0~59。
int	<i>tm_min</i>	分。范围：0~59。
int	<i>tm_hour</i>	时。范围：0~23。
int	<i>tm_mday</i>	日。范围：1~31。
int	<i>tm_mon</i>	月。范围：1~12。
int	<i>tm_year</i>	年。范围：2000~2100。
int	<i>tm_wday</i>	一周中的第几天。范围：0~6，0 为周日。

2.3.2. ql_rtc_get_time

该函数用于获取当前 RTC 时间。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_get_time(ql_rtc_time_t *tm)
```

- 参数

tm:

[Out] RTC 时间结构体指针；详见第2.3.1.2章。

- 返回值

详见第2.3.1.1章。

2.3.3. ql_rtc_print_time

该函数用于打印 RTC 时间。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_print_time(ql_rtc_time_t tm)
```

- 参数

tm:

[In] RTC 时间结构体；详见第2.3.1.2章。

- 返回值

详见第2.3.1.1章。

2.3.4. ql_rtc_set_alarm

该函数用于设置警报时间。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_set_alarm(ql_rtc_time_t* tm)
```

- 参数

tm:

[In] RTC 时间结构体指针；详见第2.3.1.2 章。

- 返回值

详见第2.3.1.1 章。

2.3.5. ql_rtc_get_alarm

该函数用于获取警报时间。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_get_alarm(ql_rtc_time_t* tm)
```

- 参数

tm:

[In] RTC 时间结构体指针；详见第2.3.1.2 章。

- 返回值

详见第2.3.1.1 章。

2.3.6. ql_rtc_enable_alarm

该函数用于启用警报功能。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_enable_alarm(unsigned char on_off)
```

- 参数

on_off:

[In] 使能参数，即是否启用警报功能。

1 启用

0 删除

- 返回值

详见第2.3.1.1 章。

2.3.7. ql_rtc_register_cb

该函数用于注册警报回调函数。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_register_cb(ql_rtc_cb cb)
```

- 参数

cb:

[In] 警报回调函数指针；详见第2.3.7.1章。

- 返回值

详见第2.3.1.1章。

2.3.7.1. ql_rtc_cb

该函数为警报回调函数。

- 函数类型

```
typedef void (*ql_rtc_cb)(void)
```

- 参数

无

- 返回值

无

2.3.8. ql_rtc_get_localtime

该函数用于获取本地时间（含时区）。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_get_localtime(ql_rtc_time_t *tm)
```

- 参数

tm:

[Out] RTC 时间结构体指针；详见第2.3.1.2 章。

- 返回值

详见第2.3.1.1 章。

2.3.9. ql_rtc_get_cfg

该函数用于获取 RTC 启动配置。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_get_cfg(ql_rtc_cfg_t *ql_rtc_cfg)
```

- 参数

ql_rtc_cfg:

[Out] RTC 启动配置结构体；详见第2.3.1.2 章。

- 返回值

详见第2.3.1.1 章。

2.3.9.1. ql_rtc_cfg_t

RTC 启动配置结构体定义如下：

```
typedef struct ql_rtc_cfg_struct {
    quec_enable_e nv_cfg;
    quec_enable_e rtc_cfg;
    quec_enable_e nwt_cfg;
}ql_rtc_cfg_t
```

- 参数

类型	参数	描述
quec_enable_e	<i>nv_cfg</i>	开机是否读取 NV 保存的时间，作为初始 RTC 值。
quec_enable_e	<i>rtc_cfg</i>	开机是否读取 RTC 寄存器的时间，作为初始 RTC 值。
quec_enable_e	<i>nwt_cfg</i>	连接基站后，是否同步基站时间到 RTC 时间。

2.3.10. ql_rtc_set_cfg

该函数用于设置 RTC 启动配置。

- 函数原型

```
ql_errcode_rtc_e ql_rtc_set_cfg(ql_rtc_cfg_t *ql_rtc_cfg)
```

- 参数

ql_rtc_cfg:

[In] RTC 启动配置结构体；详见第2.3.1.2 章。

- 返回值

详见第2.3.1.1 章。

3 示例

本章节主要介绍在 App 侧如何使用上述 API 进行 RTC 开发以及相关调试。

3.1. 开发示例

QuecOpen SDK 包中提供了 RTC API 示例，即 `ql_rtc_demo.c`，位于 `\components\ql-application\rtc` 目录下。文件中主要包含如何获取、设置、打印时间及如何设置警报功能。入口函数为 `ql_rtc_app_init()`，该函数启动 `ql_rtc_demo_thread()` 任务，如下图所示。

```
void ql_rtc_app_init()
{
    QIOSStatus err = QL_OSI_SUCCESS;
    ql_task_t rtc_task = NULL;

    err = ql_rtos_task_create(&rtc_task, 1024, APP_PRIORITY_NORMAL, "ql_rtc_demo", ql_rtc_demo_thread, NULL, 1);
    if( err != QL_OSI_SUCCESS )
    {
        QL_RTCDEMO_LOG("rtc demo task created failed");
    }
}
```

图 1: ql_rtc_app_init()

该应用程序示例将依次完成：获取当前时间并打印、设置新的时间、获取更新后的时间并打印、设置及启用警报功能。到达警报时间时，程序将自动调用 `ql_rtc_test_callback()` 回调函数。

```
static void ql_rtc_demo_thread()
{
    QL_RTCDEMO_LOG("ql rtc demo enter! \n");

    ql_rtc_time_t tm;

    //get current time
    ql_rtc_get_time(&tm);
    ql_rtc_print_time(tm);

    //2020-12-22 16:50:30 [WED]
    tm.tm_year = 2020;
    tm.tm_mon = 12;
    tm.tm_mday = 22;
    tm.tm_wday = 2;
    tm.tm_hour = 16;
    tm.tm_min = 50;
    tm.tm_sec = 30;
    ql_rtc_print_time(tm);

    //set time
    ql_rtc_set_time(&tm);

    ql_rtc_get_time(&tm);
    ql_rtc_print_time(tm);

    //set & enable alarm
    tm.tm_sec += 20;
    ql_rtc_set_alarm(&tm);
    ql_rtc_register_cb(ql_rtc_test_callback);
    ql_rtc_enable_alarm(1);

    while (1)
    {
        ql_rtc_get_time(&tm);
        QL_RTCDEMO_LOG("=====print current time=====\\r\\n");
        ql_rtc_print_time(tm);
        ql_rtc_task_sleep_s(5);
    }
} « end ql_rtc_demo_thread »
```

图 2: ql_rtc_demo_thread()

如下图所示，上述应用程序示例已在 `ql_init_demo_thread()` 线程中默认不启动。需要测试时请取消注释。

```
#ifdef QL_APP_FEATURE_GNSS
    //ql_gnss_app_init();
#endif

#ifdef QL_APP_FEATURE_SPI
    //ql_spi_demo_init();
#endif

#ifdef QL_APP_FEATURE_CAMERA
    //ql_camera_app_init();
#endif

#ifdef QL_APP_FEATURE_SPI_FLASH
    //ql_spi_flash_demo_init();
#endif

#ifdef QL_APP_FEATURE_WIFI_SCAN
    //ql_wifi_scan_app_init();
#endif

#ifdef QL_APP_FEATURE_HTTP_FOTA
    //ql_fota_http_app_init();
#endif

#ifdef QL_APP_FEATURE_DECODER
    //ql_decoder_app_init();
#endif

#ifdef QL_APP_FEATURE_KEYPAD
    //ql_keypad_app_init();
#endif

#ifdef QL_APP_FEATURE_RTC
    //ql_rtc_app_init();
#endif
```

图 3: ql_init_demo_thread()

3.2. 功能调试

LTE Standard(U)系列模块可通过移远通信 LTE OPEN EVB 板进行 RTC 功能调试。

编译固件版本烧录到模块中，使用 USB 线连接 LTE OPEN EVB 板的 USB 端口和 PC，USB AP Log Port 和 USB Diag Port 主要用于显示系统调试信息。通过 *cooltools* 或 *ArmLogel* 工具抓取 log 后，可以查看上述应用示例的相关信息。log 的抓取方法请参考文档 [3]、[4]和[5]。

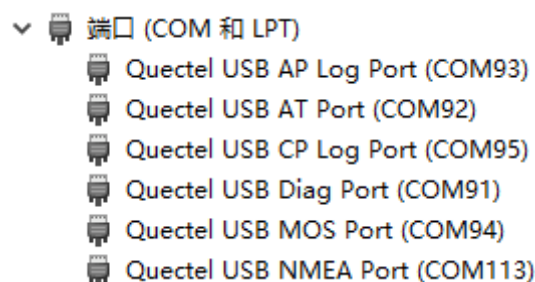


图 4: COM 设备图 1 (ECx00U 系列/EGx00U 系列/EG91xU 系列模块)



图 5: COM 设备图 2 (EC200D 系列/ECx00G 系列/EGx00G 系列模块)

备注

ECx00U 系列、EGx00U 系列和 EG91xU 系列模块使用 *cooltools* 工具抓取 log，端口使用 USB AP Log Port，抓取方法请参考文档 [3]。EC200D 系列、ECx00G 系列和 EGx00G 系列模块使用 *ArmLogel* 工具抓取 log，端口使用 USB DIAG Port，抓取方法请参考文档 [4]和[5]。

模块开机后自动启动 *ql_rtc_app_init()*。USB AP Log Port 输出的调试信息如下图所示：

Index	Received	Tick	Level	
257	11:11:18.351	15298	RTCA/I	RTC store nv length/6
819	11:11:21.558	828	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 64] ql rtc demo enter!
820	11:11:21.558	830	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 17:03:18 [TUS]
821	11:11:21.558	830	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:30 [TUS]
822	11:11:21.558	831	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:30 [TUS]
826	11:11:21.558	836	RTCA/I	RTC set alarm day/7661 hour/16 min/50 sec/50
827	11:11:21.558	861	RTCA/I	RTC store nv length/70
828	11:11:21.558	866	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
829	11:11:21.581	867	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:30 [TUS]
832	11:11:21.649	2939	RTCA/D	RTC intr 0x00001000
841	11:11:26.445	17105	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
842	11:11:26.504	17105	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:34 [TUS]
1037	11:11:31.436	33490	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
1038	11:11:31.488	33490	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:39 [TUS]
1046	11:11:36.444	49873	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
1047	11:11:36.503	49873	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:44 [TUS]
1094	11:11:41.449	721	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
1095	11:11:41.449	722	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:49 [TUS]
1096	11:11:41.449	838	QOPN/I	[ql_RTCDEMO][ql_rtc_test_callback, 49] ql rtc test callback come in!
1097	11:11:41.449	839	QOPN/I	[ql_RTCDEMO][ql_rtc_test_callback, 57] =====callback print alarm time=====
1098	11:11:41.449	839	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:50 [TUS]
1099	11:11:41.449	839	RTCA/I	RTC unset alarm
1100	11:11:41.505	862	RTCA/I	RTC store nv length/6
1124	11:11:46.440	17106	QOPN/I	[ql_RTCDEMO][ql_rtc_demo_thread, 97] =====print current time=====
1125	11:11:46.501	17106	QOPN/I	[ql_RTC][ql_rtc_print_time, 219] 2020-12-22 16:50:54 [TUS]

图 6: RTC Log 信息

4 附录 参考文档及术语缩写

表 3: 参考文档

文档名称
[1] Quectel_LTE_Standard(U)系列_QuecOpen_CSDK_快速开发指导
[2] Quectel_EC200D 系列_QuecOpen_CSDK_快速开发指导
[3] Quectel_ECx00U&EGx00U&EG91xU 系列_QuecOpen_Log_抓取指导
[4] Quectel_EC200D 系列_Log 抓取指导
[5] Quectel_ECx00G&EGx00G 系列_QuecOpen_Log 抓取指导

表 4: 术语缩写

缩写	英文全称	中文全称
AP	Application Processor	应用处理器
API	Application Programming Interface	应用程序编程接口
App	Application	应用程序
EVB	Evaluation Board	评估板
IoT	Internet of Things	物联网
LTE	Long-Term Evolution	长期演进
RTC	Real-Time Clock	实时时钟
RTOS	Real-Time Operating System	实时操作系统
SDK	Software Development Kit	软件开发工具包
USB	Universal Serial Bus	通用串行总线